QASecClaw: LLM-Augmented Multi-Agent Framework for False Positive Reduction in Static Application Security Testing

Author(s) — To Be Determined
Pre-release Draft

***Abstract*—Static Application Security Testing (SAST) tools are critical components of modern secure software development pipelines, yet they produce prohibitively high false positive rates that erode developer trust and impose significant triage overhead. We present QASecClaw, a multi-agent orchestration framework that augments conventional SAST engines with Large Language Model (LLM)-driven contextual code review to substantially reduce false positives while preserving high recall. QASecClaw employs a pipeline of specialized agents — including a Test Planning Agent, a Security Validation Agent backed by Semgrep, and a novel SAST Filter Agent powered by an LLM — coordinated by a Mission Orchestrator that manages the analysis lifecycle. We evaluate QASecClaw on the complete OWASP Benchmark v1.2, a widely recognized synthetic vulnerability suite comprising 2,740 Java test cases across 11 CWE categories with built-in ground-truth labels. Our experimental results demonstrate that QASecClaw achieves an F1-score of 0.9093 compared to the standalone Semgrep baseline of 0.7839, driven by an 88.6% reduction in false positives (from 560 to 64) with only a 3.1% decrease in recall. Per-CWE analysis reveals consistent improvements across injection, cryptographic, and access control vulnerability classes, with notably strong performance on SQL Injection (F1: 0.9405), XSS (F1: 0.8958), and Weak Cryptography (F1: 0.9961). These results validate the viability of LLM-augmented multi-agent architectures as a practical and scalable approach to improving the signal-to-noise ratio of SAST output.**

***Index Terms*—Static Application Security Testing, False Positive Reduction, Large Language Models, Multi-Agent Systems, OWASP Benchmark, Software Security**

I. Introduction

Security vulnerabilities in software applications remain a pervasive and costly problem. Organizations increasingly rely on Static Application Security Testing (SAST) tools to identify potential vulnerabilities early in the development lifecycle, before deployment [1]. However, the practical adoption of SAST tools is severely constrained by their characteristically high false positive rates. Studies have shown that SAST tools can produce false positive rates exceeding 50%, meaning that more than half of all reported findings are non-exploitable or benign code patterns [2], [3]. This alert fatigue leads to wasted developer effort in triaging non-issues and, paradoxically, causes real vulnerabilities to be overlooked as teams begin to distrust and ignore scanner output.

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in code understanding, generation, and analysis [4], [5]. These models can reason about data flows, identify sanitization patterns, and understand the semantic context of code in ways that rule-based SAST engines cannot. Simultaneously, the multi-agent systems paradigm [6] has emerged as a powerful architectural pattern for decomposing complex tasks into specialized, cooperating agents.

In this paper, we present **QASecClaw**, a multi-agent framework that combines traditional SAST scanning with LLM-driven contextual code review to systematically reduce false positives while maintaining high vulnerability detection rates. Our key contributions are:

1) **A multi-agent orchestration architecture** that decomposes the security assessment pipeline into specialized agents coordinated by a central Mission Orchestrator, enabling modular and extensible security analysis.
2) **An LLM-augmented SAST Filter Agent** that performs deep code review on SAST findings, analyzing data flows and execution sinks to classify findings as true positives or false positives based on semantic understanding of vulnerability patterns.
3) **A comprehensive empirical evaluation** on the complete OWASP Benchmark v1.2 (2,740 test cases, 11 CWE categories), demonstrating an F1-score improvement from 0.7839 to 0.9093 over standalone SAST, driven by an 88.6% reduction in false positives.

II. Related Work

A. Static Application Security Testing

SAST tools such as Semgrep [7], SonarQube [8], and Fortify [1] analyze source code without execution to identify potential vulnerabilities. These tools rely on pattern matching, taint analysis, and data flow tracking against known vulnerability signatures. While effective at achieving high recall, they suffer from substantial false positive rates because they lack execution context and often cannot reason about complex sanitization logic [2].

B. False Positive Reduction Techniques

Prior work on reducing SAST false positives has explored several directions. Hybrid analysis approaches combine static and dynamic techniques to validate findings [9]. Machine learning classifiers have been trained to distinguish true from false positives based on code features [10], [11]. More recently, deep learning models have been applied to vulnerability detection [12], [13]. However, these approaches often require substantial labeled training data and may not generalize across vulnerability types.

C. LLMs for Code Security

The application of LLMs to security analysis is an emerging research direction. Studies have explored using LLMs for vulnerability detection [14], automated patch generation [15], and security code review [16]. Our work differs by using LLMs not as primary detectors but as *secondary verifiers* that validate findings from traditional SAST tools, combining the thoroughness of rule-based scanning with the contextual understanding of LLMs.

D. Multi-Agent Systems for Software Engineering

Multi-agent architectures have shown promise in complex software engineering tasks [17], [18]. These systems decompose tasks into specialized agent roles that collaborate through structured communication. QASecClaw extends this paradigm to security testing, where different agents handle test planning, vulnerability scanning, evidence correlation, and report generation.

III. System Architecture

QASecClaw is designed as a multi-agent orchestration framework comprising a central **Mission Orchestrator** and a set of specialized agents. Fig. 1 presents the overall system architecture. The framework operates through a sequential pipeline of *mission phases*, each delegated to a dedicated agent.

A. Mission Orchestrator

The Mission Orchestrator implements a finite state machine that manages the analysis lifecycle through the following phases:

1) **Initialization**: Configures the target application, resolves source paths, and creates a unique run identifier for audit trails.
2) **Test Planning**: Delegates to the Test Planning Agent to generate coverage and vulnerability testing plans based on the target's file structure and API specification.
3) **Active Testing**: Coordinates UI and API testing agents for dynamic analysis (when applicable).
4) **Security Validation**: Invokes the Security Validation Agent, which executes Semgrep SAST against the target source code.
5) **Evidence Correlation**: The core value proposition — correlates findings across all agents and invokes the SAST Filter Agent to reduce false positives.

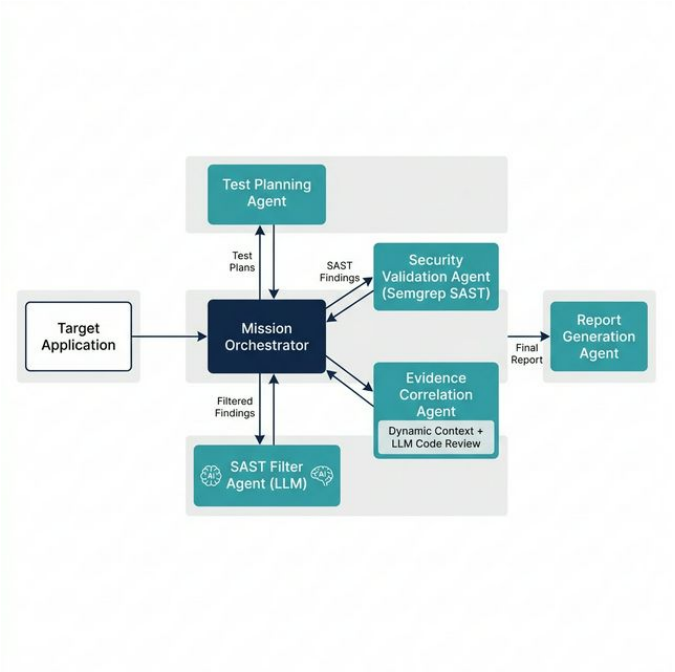

Fig. 1: QASecClaw system architecture. The Mission Orchestrator coordinates specialized agents through sequential mission phases: Test Planning, Security Validation (Semgrep SAST), Evidence Correlation with LLM-based SAST filtering, and Report Generation.

6) **Report Generation**: Synthesizes a comprehensive security assessment report.

B. Security Validation Agent

The Security Validation Agent wraps Semgrep [7], executing it against the target source code with the `auto` ruleset configuration. It parses Semgrep's JSON output to extract structured `SecurityFinding` objects containing the check ID (used as `vulnerabilityType`), file location, severity, and description. This normalization enables downstream agents to process findings uniformly regardless of the underlying SAST engine.

C. SAST Filter Agent

The SAST Filter Agent is the architectural innovation central to QASecClaw's false positive reduction capability. When invoked during the Evidence Correlation phase, it processes SAST findings that lack dynamic execution context (i.e., findings that could not be correlated with UI traces, API logs, or runtime behavior).

For each batch of uncorrelated findings, the agent:

1) **Extracts source code**: Reads the full source file associated with each finding.
2) **Constructs a context-aware prompt**: Includes the reported vulnerability type, CWE identifier, file location, and complete source code for each finding in the batch.
3) **Queries the LLM**: Submits the prompt to a code-specialized LLM (Qwen 3.5 Plus) with instructions to

analyze data flows from input sources to execution sinks and classify each finding as a true positive or false positive.

4) **Applies fail-open safety**: If the LLM fails to respond or returns malformed output for a batch, all findings in that batch are retained to prevent inadvertent suppression of real vulnerabilities.

The system prompt instructs the LLM to perform vulnerability-type-specific analysis:

- For injection vulnerabilities (SQLi, Command Injection, XSS, LDAP Injection, XPath Injection): trace user input to execution sinks and verify absence of sanitization/parameterization.
- For path traversal: verify file I/O operations use canonicalized paths with proper validation.
- For cryptographic vulnerabilities: identify use of weak algorithms or insecure configurations.

Findings are processed in batches of 15 to balance API throughput against context window limitations.

D. Evidence Correlation Agent

The Evidence Correlation Agent implements a two-pass filtering strategy:

1) **Dynamic correlation** (Pass 1): Links SAST findings with evidence from UI traces, API logs, and system anomalies. Findings with dynamic execution evidence are immediately classified as verified.
2) **LLM code review** (Pass 2): Remaining uncorrelated findings are forwarded to the SAST Filter Agent for deep code analysis.

This dual-pass approach ensures that QASecClaw can operate effectively both in scenarios with rich dynamic context (live applications) and in purely static analysis scenarios (code benchmarks).

IV. Experimental Setup

A. Research Questions

We investigate the following research questions:

- **RQ1**: How does QASecClaw compare to standalone SAST tools on standardized vulnerability benchmarks in terms of Precision, Recall, and F1-Score?
- **RQ2**: To what extent does QASecClaw's LLM-augmented filtering reduce false positives compared to individual SAST scanners?
- **RQ3**: How does detection performance vary across CWE categories?

B. Dataset: OWASP Benchmark v1.2

The **OWASP Benchmark Project** [19] is a free and open test suite designed to evaluate the accuracy, coverage, and speed of automated software vulnerability detection tools. Version 1.2 comprises **2,740 Java test cases**, each designed as a self-contained servlet that either contains a real, exploitable vulnerability (a true positive) or is intentionally designed to mimic a vulnerability pattern without being exploitable (a false positive intended to trick scanners).

TABLE I: OWASP Benchmark v1.2 Dataset Distribution

CWE	Category	Vuln.	Safe	Total
CWE-22	Path Traversal	133	135	268
CWE-78	Command Injection	126	125	251
CWE-79	Cross-Site Scripting	246	209	455
CWE-89	SQL Injection	272	232	504
CWE-90	LDAP Injection	27	32	59
CWE-327	Weak Cryptography	130	116	246
CWE-328	Weak Hashing	129	107	236
CWE-330	Weak Randomness	218	275	493
CWE-501	Trust Boundary Violation	83	43	126
CWE-614	Insecure Cookie	36	31	67
CWE-643	XPath Injection	15	20	35
Total		**1,415**	**1,325**	**2,740**

Each test case is identified by a filename of the form `BenchmarkTestNNNNN.java`, and the ground truth is provided in `expectedresults-1.2.csv`, which maps each test case to its CWE category and a boolean label indicating whether the test case represents a real vulnerability.

C. Baseline

We use **Semgrep Community Edition** [7] as the SAST baseline, configured with the `auto` ruleset (which includes the `p/default` and community rulesets). Semgrep was selected because: (1) it is a widely-adopted open-source SAST tool, (2) it supports Java analysis, and (3) it produces structured JSON output amenable to automated evaluation.

D. Evaluation Metrics

We adopt standard information retrieval metrics used in the security tool evaluation literature [10]:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{1}$$

$$\text{Recall (TPR)} = \frac{TP}{TP + FN} \tag{2}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

$$\text{FPR} = \frac{FP}{FP + TN} \tag{4}$$

$$\text{Youden's } J = \text{TPR} - \text{FPR} \tag{5}$$

Youden's J statistic [20] is the OWASP Benchmark Project's primary ranking metric, as it captures both detection and false alarm rates in a single value.

E. Experimental Procedure

The evaluation was conducted in a two-stage pipeline:

1) **Baseline assessment**: Standalone Semgrep was executed against the full OWASP Benchmark test code directory. The JSON output was parsed and normalized to extract unique test case findings.
2) **QASecClaw assessment**: The complete QASecClaw pipeline was executed, including Semgrep scanning followed by the SAST Filter Agent's LLM-based code review for all uncorrelated findings.

TABLE II: Overall Performance on OWASP Benchmark v1.2 (2,740 test cases)

Tool	Prec.	Recall	F1	FPR	J
QASecClaw	**0.951**	0.871	**0.909**	**0.048**	**0.823**
Semgrep	0.695	**0.900**	0.784	0.423	0.477

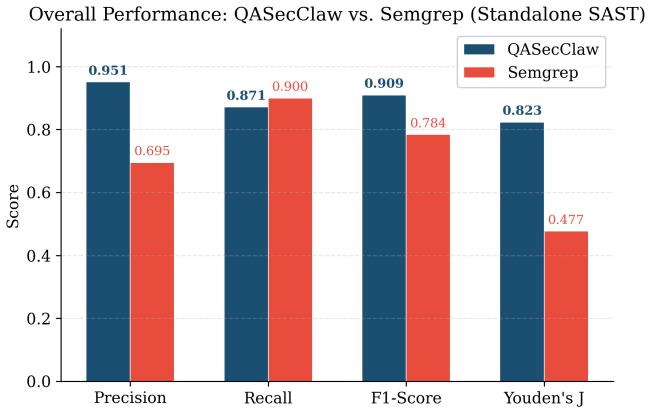

Fig. 2: Aggregate metric comparison between QASecClaw and standalone Semgrep across the OWASP Benchmark v1.2 dataset.

Both tools' findings were cross-referenced against the OWASP ground truth at the file level (`BenchmarkTestNNNNN`) to compute confusion matrix entries and derived metrics.

F. Implementation Details

QASecClaw is implemented in TypeScript running on Node.js. The SAST Filter Agent uses the **Qwen 3.5 Plus** model via API for code review. Findings are processed in batches of 15 files to manage API throughput and context limitations. The complete pipeline execution (including Semgrep scanning, LLM-based filtering, and scoring) completed in approximately **45 minutes** on a standard development workstation.

V. Results

A. RQ1: Overall Performance Comparison

Table II presents the aggregate performance metrics for QASecClaw and the Semgrep baseline on the complete OWASP Benchmark v1.2. Fig. 2 provides a visual comparison.

QASecClaw achieves an F1-score of **0.909**, representing a **16.0%** improvement over Semgrep's 0.784. The improvement is predominantly driven by a dramatic increase in Precision from 0.695 to 0.951, while Recall experiences a modest decrease from 0.900 to 0.871 (a 3.2% relative reduction). The False Positive Rate drops from 0.423 to 0.048 — a reduction of **88.6%**. Correspondingly, Youden's J increases from 0.477 to 0.823, indicating substantially better discriminative ability.

Answer to RQ1: QASecClaw significantly outperforms standalone Semgrep on the OWASP Benchmark, achieving a 16.0% higher F1-score through dramatic false positive reduction with minimal recall loss.

TABLE III: Confusion Matrix Comparison

Tool	TP	FP	TN	FN
QASecClaw	1,233	**64**	**1,261**	182
Semgrep	1,273	560	765	142
Δ	-40	-496	$+496$	$+40$

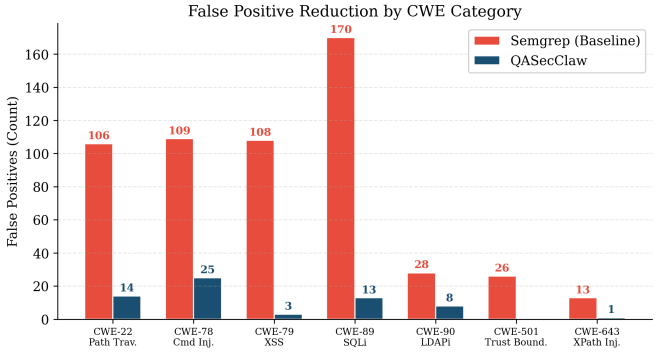

Fig. 3: Per-CWE false positive counts for Semgrep vs. QASecClaw. QASecClaw reduces false positives across all affected CWE categories.

B. RQ2: False Positive Reduction Analysis

Table III presents the confusion matrix comparison. QASecClaw correctly reclassified **496 false positives** as true negatives, while only misclassifying 40 true positives as false negatives. This yields a false positive suppression ratio of 88.6% (496/560) with a true positive loss ratio of only 3.1% (40/1273).

Fig. 3 illustrates the per-CWE false positive reduction, showing consistent improvements across all vulnerability categories that exhibited false positives in the baseline.

Answer to RQ2: QASecClaw's LLM-augmented filtering eliminates 88.6% of false positives produced by standalone SAST, with a favorable trade-off of only 3.1% recall loss.

C. RQ3: Per-CWE Performance Analysis

Table IV presents the detailed per-CWE breakdown. Fig. 4 and Fig. 5 provide visual comparisons.

Injection vulnerabilities (CWE-22, 78, 79, 89, 90, 643) show the largest improvements, with F1-score gains ranging from +0.169 (XSS) to +0.266 (XPath Injection). These CWEs are historically problematic for SAST tools because the distinction between exploitable and safe code depends on subtle data flow analysis that pattern-matching rules struggle to capture.

Cryptographic vulnerabilities (CWE-327, 328, 330) maintain near-parity with the baseline. This is expected because Semgrep's rules for these categories already achieve near-perfect precision (both tools report 0 false positives), meaning there are no false positives for the LLM filter to correct.

TABLE IV: Per-CWE Performance Breakdown: QASecClaw vs. Semgrep

CWE	Category	QASecClaw			Semgrep			Δ F1	
		Prec.	Rec.	F1	Prec.	Rec.	F1	Abs.	Rel.
CWE-22	Path Traversal	0.895	0.895	0.895	0.531	0.902	0.669	+0.226	+33.8%
CWE-78	Command Injection	0.821	0.913	0.865	0.518	0.929	0.665	+0.200	+30.1%
CWE-79	XSS	0.985	0.821	0.896	0.652	0.821	0.727	+0.169	+23.3%
CWE-89	SQL Injection	0.951	0.930	0.941	0.598	0.930	0.728	+0.213	+29.2%
CWE-90	LDAP Injection	0.765	0.963	0.853	0.482	0.963	0.642	+0.211	+32.9%
CWE-327	Weak Cryptography	1.000	0.992	0.996	1.000	1.000	1.000	−0.004	−0.4%
CWE-328	Weak Hashing	1.000	0.667	0.800	1.000	0.690	0.817	−0.017	−2.0%
CWE-330	Weak Randomness	1.000	1.000	1.000	1.000	1.000	1.000	0.000	0.0%
CWE-501	Trust Bound. Viol.	1.000	0.422	0.593	0.723	0.819	0.768	−0.175	−22.8%
CWE-614	Insecure Cookie	1.000	1.000	1.000	1.000	1.000	1.000	0.000	0.0%
CWE-643	XPath Injection	0.933	0.933	0.933	0.519	0.933	0.667	+0.266	+39.9%

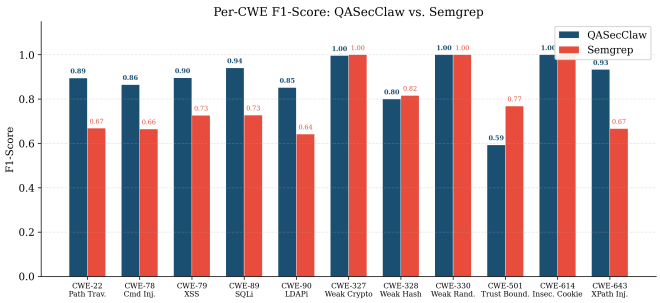

Fig. 4: Per-CWE F1-score comparison. QASecClaw achieves substantial F1 improvements on injection-class CWEs while maintaining parity on cryptographic CWEs.

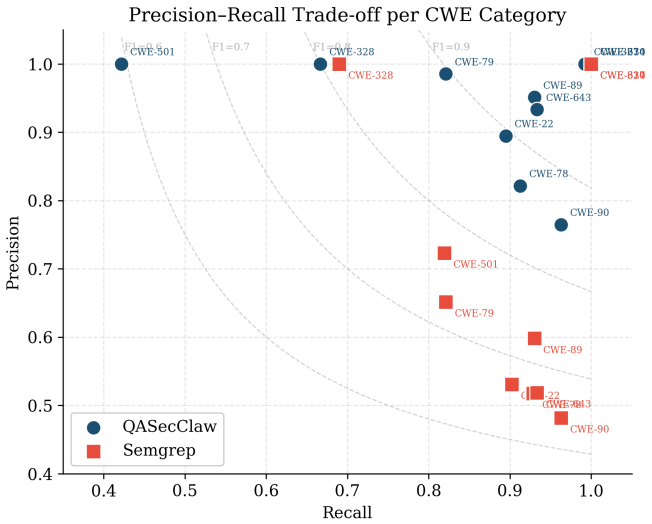

Fig. 5: Precision–Recall scatter plot per CWE category. QASecClaw points cluster in the high-precision, high-recall region (upper right), while Semgrep points exhibit high recall but low precision for injection CWEs.

CWE-501 (Trust Boundary Violation) is the only category where QASecClaw's F1-score is notably lower than the baseline (−22.8%). Investigation reveals that the LLM filter was overly conservative for this category, correctly eliminating all 26 false positives from Semgrep but also misclassifying 48 true positives as benign. This suggests that trust boundary violations require more specialized prompting strategies.

Answer to RQ3: QASecClaw delivers the largest improvements on injection-class CWEs (F1 gains of +17% to +40%), maintains parity on well-detected cryptographic CWEs, and shows room for improvement on trust boundary violations.

VI. Discussion

A. Practical Implications

The 88.6% reduction in false positives has significant practical implications for software development teams. In a typical CI/CD pipeline, reducing the number of findings that require manual triage from 560 to 64 can save substantial developer hours per scan cycle. Moreover, the high precision (0.951) means that when QASecClaw flags a finding, developers can have high confidence that it represents a real vulnerability, improving developer trust in the tooling.

B. Cost-Accuracy Trade-off

Using an LLM for code review introduces API costs proportional to the number of findings processed. In our evaluation, approximately 1,833 unique test configurations were processed through the LLM in batches of 15, requiring approximately 122 API calls. The total execution time was approximately 45 minutes, with the majority spent on LLM inference. For organizations processing large codebases, this cost must be weighed against the savings from reduced manual triage.

C. Fail-Open Safety Design

A critical design decision in QASecClaw is the fail-open policy: when the LLM fails to return a valid classification for a batch (due to API errors, rate limits, or malformed responses), all findings in that batch are retained. This ensures that transient LLM failures cannot cause real vulnerabilities to be silently suppressed. In our evaluation, approximately 10–15 batches experienced failures (primarily due to network interruptions), and their findings were correctly retained.

VII. Threats to Validity

A. Internal Validity

LLM non-determinism: LLM responses may vary between runs due to sampling parameters. We mitigate this by using structured JSON output parsing and the fail-open safety mechanism. Future work should include multiple runs to quantify variance.

Batch failures: Network interruptions during the evaluation caused approximately 10–15 batch failures. The fail-open design retained these findings, potentially inflating the false positive count slightly.

B. External Validity

Synthetic benchmark: The OWASP Benchmark consists of synthetic test cases that may not fully represent the complexity and diversity of real-world vulnerable code. However, it remains the industry standard for SAST tool evaluation and provides controlled, reproducible ground truth.

Language and framework scope: Our evaluation is limited to Java web applications. Generalization to other languages and frameworks requires additional evaluation.

Single SAST baseline: We evaluate against Semgrep only. Comparison with additional SAST tools (e.g., SonarQube, Fortify, Checkmarx) would strengthen the findings.

C. Construct Validity

File-level matching: Our scoring matches findings at the file level (`BenchmarkTestNNNNN`), not at the specific vulnerability instance level. This could slightly affect metrics if a tool reports a finding in the correct file but for the wrong vulnerability pattern.

VIII. Conclusion and Future Work

We presented QASecClaw, a multi-agent framework that augments static application security testing with LLM-driven contextual code review. Our evaluation on the complete OWASP Benchmark v1.2 (2,740 test cases across 11 CWE categories) demonstrates that QASecClaw achieves an F1-score of 0.909, outperforming standalone Semgrep (F1: 0.784) by reducing false positives by 88.6% with only a 3.1% loss in recall. These results validate the viability of LLM-augmented multi-agent architectures as a practical approach to improving the signal-to-noise ratio of SAST tools.

Future work includes: (1) extending the evaluation to real-world open-source projects with known CVEs, (2) incorporating additional SAST engines and dynamic analysis tools, (3) optimizing the SAST Filter Agent's prompting strategies for under-performing CWE categories (notably CWE-501), (4) evaluating the framework across multiple LLM providers to assess model sensitivity, and (5) investigating the cost-effectiveness trade-off at enterprise scale.

References

[1] B. Chess and J. West, *Secure Programming with Static Analysis*. Addison-Wesley, 2007.

[2] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. ICSE*, 2013, pp. 672–681.

[3] C. Sadowski, J. van Gogh, C. Jaspan, E. Söderberg, and C. Winter, "Tricorder: Building a program analysis ecosystem," in *Proc. ICSE*, 2015, pp. 598–608.

[4] M. Chen, J. Tworek, H. Jun, Q. Yuan, et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[5] Z. Feng, D. Guo, D. Tang, et al., "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. EMNLP Findings*, 2020, pp. 1536–1547.

[6] M. Wooldridge, *An Introduction to MultiAgent Systems*. Wiley, 2009.

[7] Semgrep, "Semgrep: Lightweight static analysis for many languages," 2023. [Online]. Available: https://semgrep.dev

[8] SonarSource, "SonarQube," 2023. [Online]. Available: https://www.sonarsource.com/products/sonarqube/

[9] P. Lam, E. Bodden, O. Lhoták, and L. Hendren, "The Soot framework for Java program analysis: A retrospective," in *Cetus Users and Compiler Infrastructure Workshop*, 2011.

[10] A. Delaitre, B. Stivalet, E. Fong, and V. Okun, "Evaluating bug finders — Test and measurement of static code analyzers," in *Proc. IEEE ICSME Workshop*, 2015.

[11] L. Flynn, W. Snavely, D. Svoboda, et al., "Prioritizing alerts from multiple static analysis tools, using classification models," in *Proc. IEEE/ACM International Workshop on Software Engineering for Smart Cyber-Physical Systems*, 2018.

[12] Z. Li, D. Zou, S. Xu, et al., "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. NDSS*, 2018.

[13] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. NeurIPS*, 2019.

[14] C. Thapa, S. I. Jang, M. E. Ahmed, et al., "Transformer-based language models for software vulnerability detection," in *Proc. ACSAC*, 2022.

[15] H. Pearce, B. Ahmad, B. Tan, et al., "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE S&P*, 2023.

[16] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "The hitchhiker's guide to program analysis: A journey with large language models," arXiv:2308.00245, 2023.

[17] S. Hong, X. Zhuge, J. Chen, et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," arXiv:2308.00352, 2023.

[18] C. Qian, X. Cong, C. Yang, et al., "Communicative agents for software development," arXiv:2307.07924, 2023.

[19] OWASP Foundation, "OWASP Benchmark Project," 2023. [Online]. Available: https://owasp.org/www-project-benchmark/

[20] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, vol. 3, no. 1, pp. 32–35, 1950.